Übungsblatt 2

Donnerstag, 13. November 2025 22:28

Yunis Drescher, 827130 Joshua Neunert, 822456 Konrad Gose, 828440

10/11

Aufgabe 2.2: Scheduling

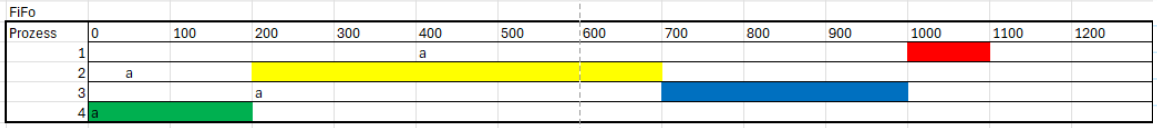
5/5 a) In der Übung wurde die FiFo-Scheduling-Strategie in den **Sched-Sim** Simulator implementiert. Erweitern Sie **schedule.c** um die Implementierung der Round-Robin-, Shortest-Job-First- und Fixed-Priority-Scheduling-Strategie. Erweitern Sie falls nötig die Implementierung der Run-Queue in **queue.c** um die benötigten Funktionen. (5 Punkte)

2/3 b) Erstellen Sie für die Algorithmen Round-Robin, Shortest-Job-First und Fixed-Priority je ein Ganttdiagramm, wobei die Kosten für Scheduler- und Dispatcher vernachlässigt werden sollen. Übernehmen Sie die Laufzeit und Ankunftszeiten der Prozess aus der **main.c**.

Hinweis: Verwenden Sie für ihre Zeichnung ein Quantum von 100ms. (3 Punkte)

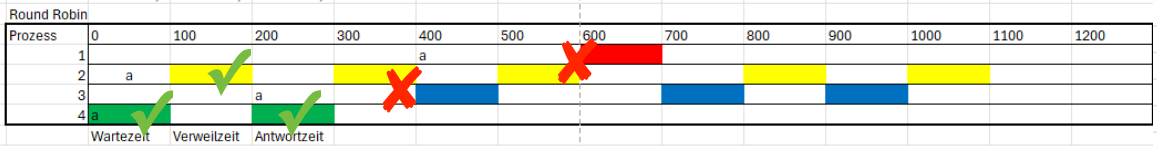
3/3 c) Bewerten Sie die **Effizienz** der drei Scheduling-Strategien in Bezug auf die mittlere Wartezeit, die mittlere Verweildauer und die mittlere Antwortzeit. Beziehen Sie sich dabei auf die in Aufgabe b erstellten Ablaufdiagramme. (3 Punkte)

FiFo



	Wartezeit	Verweilzeit	Antwortzeit
1	600	700	600
2	150	650	150
3	500	800	500
4	0	200	0
Durchschnitt	312,5	587,5	312,5

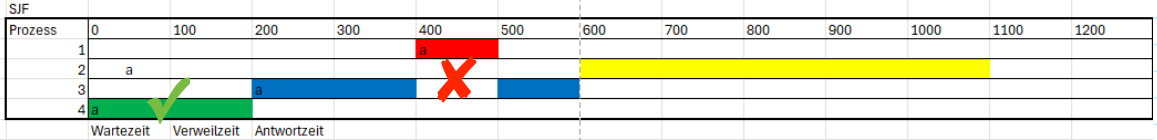
Round Robin



	Wartezeit	Verweilzeit	Antwortzeit
1	200	300	200
2	550	1050	50
3	500	800	200
4	100	300	0
Durchschnitt	337,5	612,5	112,5

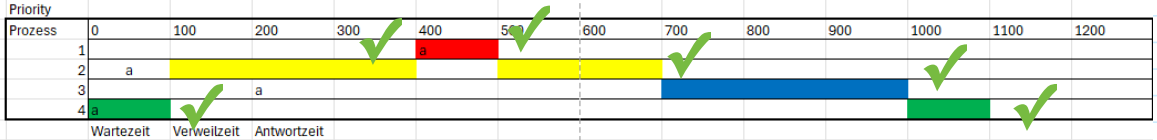
Folgefehler

SJF



	Wartezeit	Verweilzeit	Antwortzeit
1	0	100	0
2	550	1050	550
3	100	400	0
4	0	200	0
Durchschnitt	162,5	437,5	137,5

Priority



	Wartezeit	Verweilzeit	Antwortzeit
1	0	100	0
2	150	650	50
3	500	800	500
4	900	1100	0
Durchschnitt	387,5	662,5	137,5

	Wartezeit	Verweilzeit	Antwortzeit	
1. Platz	SJF = 162,5	SJF = 437,5	RR = 112,5	1 Punkt
2. Platz	FiFo = 312,5	FiFo = 587,5	SJF / Prio	2 Punkte
3. Platz	RR= 337,5	RR = 612,5	= 137,5	3 Punkte
4. Platz	Prio = 387,5	Prio = 662,5	FiFo = 312,5	4 Punkte

Insgesamt		
1. Platz	SJF	4 Punkte
2. Platz	RR	7 Punkte
3. Platz	FiFo	8 Punkte
4. Platz	Prio	10 Punkte

Folgefehler

zu b)

Round Robin: P3 wird bei Ankunft bei 200 ms vor P2 eingeordnet und startet ab 300 ms. -0,5 Analog wird bei der Ankunft von P1 vorgegangen, wodurch P1 bereits ab 500 ms startet.

Die richtige Reihenfolge lautet also: P4, P2, P4, P3, P2, P1, …

SJF: P3 wird nicht durch die Ankunft von P1 bei 400 ms unterbrochen. -0,5 SJF arbeitet nicht-verdrängend.

a) 5/5

queue.c

```c
#include <queue.h>
#include <stdlib.h>
#include <stddef.h>
#include <process.h>
#include <schedule.h>
#include <internal/misc.h>

int queue_enqueue_rr(process_queue_t *queue, process_t
*process);
int queue_enqueue_sjf(process_queue_t *queue, process_t
*process);
int queue_enqueue_prio(process_queue_t *queue, process_t
*process);

void queue_set_functions(void)
{
    // setting queue functions
    global_state.queue_functions.queue_init = queue_init;
    global_state.queue_functions.queue_enqueue =
queue_enqueue;
    global_state.queue_functions.queue_destroy =
queue_destroy;
    global_state.queue_functions.queue_length = queue_length;
    global_state.queue_functions.queue_get_process_at =
queue_get_process_at;

    // If you want that new processes are added in a different
way for one scheduler
    if (schedule == schedule_sjf)
    {
        global_state.queue_functions.queue_enqueue =
queue_enqueue_sjf;
    }
    else if (schedule == schedule_prio)
    {
        global_state.queue_functions.queue_enqueue =
queue_enqueue_prio;
    }
    else if (schedule == schedule_rr)
    {
        global_state.queue_functions.queue_enqueue =
queue_enqueue_rr;
    }

    return;
}

int queue_init(process_queue_t **queue)
{
    *queue = (process_queue_t
```

```
*)malloc(sizeof(process_queue_t));
    if (*queue == NULL)
    {
        return -1;
    }
    (*queue)->head = NULL;
    (*queue)->tail = NULL;
    (*queue)->length = 0;
    return 0;
}

// Default
int queue_enqueue(process_queue_t *queue, process_t *process)
{
    process_node_t *new_node = (process_node_t
*)malloc(sizeof(process_node_t));
    if (new_node == NULL)
    {
        return -1;
    }
    new_node->process = process;
    new_node->prev = queue->tail;
    new_node->next = NULL;

    if (queue->tail)
    {
        queue->tail->next = new_node;
    }
    else
    {
        queue->head = new_node;
    }
    queue->tail = new_node;
    queue->length++;
    return 0;
}

// Round Robin
int queue_enqueue_rr(process_queue_t *queue, process_t
*process)
{
    // args validation
    if (!queue || !process)
    {
        return -1;
    }

    process_node_t *new_node = (process_node_t
*)malloc(sizeof(process_node_t));
    if (new_node == NULL)
```

```
    {
        return -1;
    }

    new_node->process = process;
    new_node->prev = queue->tail;
    new_node->next = NULL;

    if (queue->tail)
    {
        // Add to the end
        queue->tail->next = new_node;
    }
    else
    {
        // Default set as head
        queue->head = new_node;
    }

    queue->tail = new_node;
    queue->length++;
    return 0;
}

// Shortest-Job-First
int queue_enqueue_sjf(process_queue_t *queue, process_t
*process)
{
    // args validation
    if (!queue || !process)
    {
        return -1;
    }

    process_node_t *new_node = (process_node_t
*)malloc(sizeof(process_node_t));
    if (new_node == NULL)
    {
        return -1;
    }

    new_node->process = process;
    new_node->prev = NULL;
    new_node->next = NULL;

    if (queue->head == NULL)
    {
        // Insert as Head
        queue->head = new_node;
        queue->tail = new_node;
```

```
    }
    else
    {
        process_node_t *current = queue->head;
        while (current != NULL && current->process->burst_time
<= process->burst_time)
        {
            current = current->next;
        }
        if (current == queue->head)
        {
            new_node->next = queue->head;
            queue->head->prev = new_node;
            queue->head = new_node;
        }
        else if (current == NULL)
        {
            new_node->prev = queue->tail;
            queue->tail->next = new_node;
            queue->tail = new_node;
        }
        else
        {
            new_node->next = current;
            new_node->prev = current->prev;
            current->prev->next = new_node;
            current->prev = new_node;
        }
    }
    queue->length++;
    return 0;
}

// Fixed-Priority
int queue_enqueue_prio(process_queue_t *queue, process_t
*process)
{
    process_node_t *new_node = (process_node_t
*)malloc(sizeof(process_node_t));
    if (new_node == NULL)
    {
        return -1;
    }

    new_node->process = process;
    new_node->prev = NULL;
    new_node->next = NULL;

    if (queue->head == NULL)
    {
```

```
        queue->head = new_node;
        queue->tail = new_node;
    }
    else
    {
        process_node_t *current = queue->head;
        while (current != NULL && current->process->priority
<= process->priority)
        {
            current = current->next;
        }
        if (current == queue->head)
        {
            new_node->next = queue->head;
            queue->head->prev = new_node;
            queue->head = new_node;
        }
        else if (current == NULL)
        {
            new_node->prev = queue->tail;
            queue->tail->next = new_node;
            queue->tail = new_node;
        }
        else
        {
            new_node->next = current;
            new_node->prev = current->prev;
            current->prev->next = new_node;
            current->prev = new_node;
        }
    }
    queue->length++;
    return 0;
}

int queue_dequeue(process_queue_t *queue, process_t **process)
{
    if (queue->head == NULL)
    {
        return -1;
    }
    process_node_t *node_to_remove = queue->head;
    *process = node_to_remove->process;
    queue->head = node_to_remove->next;

    if (queue->head)
    {
        queue->head->prev = NULL;
    }
    else
```

```
    {
        queue->tail = NULL;
    }
    free(node_to_remove);
    queue->length--;
    return 0;
}

int queue_length(process_queue_t *queue)
{
    return queue->length;
}

int queue_get_process_at(process_queue_t *queue, process_t
**process, int index)
{
    if (index < 0 || index >= (int)queue->length)
    {
        return -1;
    }
    process_node_t *current = queue->head;
    for (int i = 0; i < index; i++)
    {
        current = current->next;
    }
    *process = current->process;
    return 0;
}

int queue_destroy(process_queue_t *queue)
{
    process_node_t *current = queue->head;
    while (current != NULL)
    {
        process_node_t *next = current->next;
        free(current);
        current = next;
    }
    free(queue);
    return 0;
}
```

schedule.c

```c
#include <schedule.h>
#include <process.h>
#include <queue.h>
#include <internal/misc.h>

int schedule_fifo(process_t *current_process, process_t
**next_process) {
  if(current_process && current_process->state == READY) {
    *next_process = current_process;
    return 0;
  }

  if (queue_length(current_core->run_queue) < 1) {
    return -1;
  }

  return queue_dequeue(current_core->run_queue, next_process);
}

// Round Robin
int schedule_rr(process_t *current_process, process_t
**next_process) {
  int ret;
  process_t *next = NULL;

  // first element in the run queue
  ret = queue_dequeue(current_core->run_queue, &next);

  if (ret == 0) {

    // append the current process back to the run queue if
it's still READY
    if (current_process && current_process->state == READY) {
      int enq_ret = queue_enqueue(current_core->run_queue,
current_process);
      if (enq_ret != 0) {
        return enq_ret;
      }
    }

    // assign the next process to run
    *next_process = next;
    return 0;

  } else {
    // empty queue

    // let the current process continue if it's still READY
    if (current_process && current_process->state == READY) {
      *next_process = current_process;
```

```
      return 0;
    }

    // queue is empty
    return -1;
  }
}

// Shortest Job First
int schedule_sjf(process_t *current_process, process_t
**next_process) {
  int len = 0;
  // If a process is running continue
  if(current_process && current_process->state == READY) {
    *next_process = current_process;
    return 0;
  }
  // If no process is running find next
  if ((len = queue_length(current_core->run_queue)) < 1) {
    return -1;
  }

  process_t *shortest_job = NULL;
  process_t *temp_process = NULL;
  int ret = 0;

  // find shortest job in the run queue
  for (int i = 0; i < len; i++) {
    ret = queue_dequeue(current_core->run_queue,
&temp_process);
    if (ret != 0) {
      return ret;
    }

    // Check if process is new shortest job
    if (shortest_job == NULL || temp_process->burst_time <
shortest_job->burst_time) {
      if (shortest_job != NULL) {
        ret = queue_enqueue(current_core->run_queue,
shortest_job);
        if (ret != 0) {
          return ret;
        }
      }
      shortest_job = temp_process;
    } else {
      ret = queue_enqueue(current_core->run_queue,
temp_process);
      if (ret != 0) {
        return ret;
```

```
      }
    }
  }

  *next_process = shortest_job;
  return 0;
}

// Fixed Priority Scheduling
int schedule_prio(process_t *current_process, process_t
**next_process) {
  int len = queue_length(current_core->run_queue);
  int ret = 0;
  process_t *highest_in_queue = NULL;
  process_t *temp_process = NULL;

  // find highest priority process in the run queue
  if (len > 0) {
    for (int i = 0; i < len; i++) {
      ret = queue_dequeue(current_core->run_queue,
&temp_process);
      if (ret != 0) return ret;


      if (highest_in_queue == NULL || temp_process->priority >
highest_in_queue->priority) {
        if (highest_in_queue != NULL) {
          ret = queue_enqueue(current_core->run_queue,
highest_in_queue);
          if (ret != 0) return ret;
        }
        highest_in_queue = temp_process;
      } else {
        ret = queue_enqueue(current_core->run_queue,
temp_process);
        if (ret != 0) return ret;
      }
    }
  }

  // continue with current process or switch to highest
priority from queue
  if (current_process && current_process->state == READY) {

    if (highest_in_queue == NULL) {
      // empty queue, continue current process
      *next_process = current_process;
      return 0;
    }
```

```
    // Compare priorities
    if (current_process->priority >= highest_in_queue-
>priority) {
      *next_process = current_process;
      ret = queue_enqueue(current_core->run_queue,
highest_in_queue);
      return ret;
    } else {
      // Switch to higher priority process
      *next_process = highest_in_queue;
      ret = queue_enqueue(current_core->run_queue,
current_process);
      return ret;
    }

  } else {
    // no current process, run highest priority from queue
    if (highest_in_queue == NULL) {
      // empty queue
      return -1;
    }
    // Start the highest priority process from the queue.
    *next_process = highest_in_queue;
    return 0;
  }
}
```